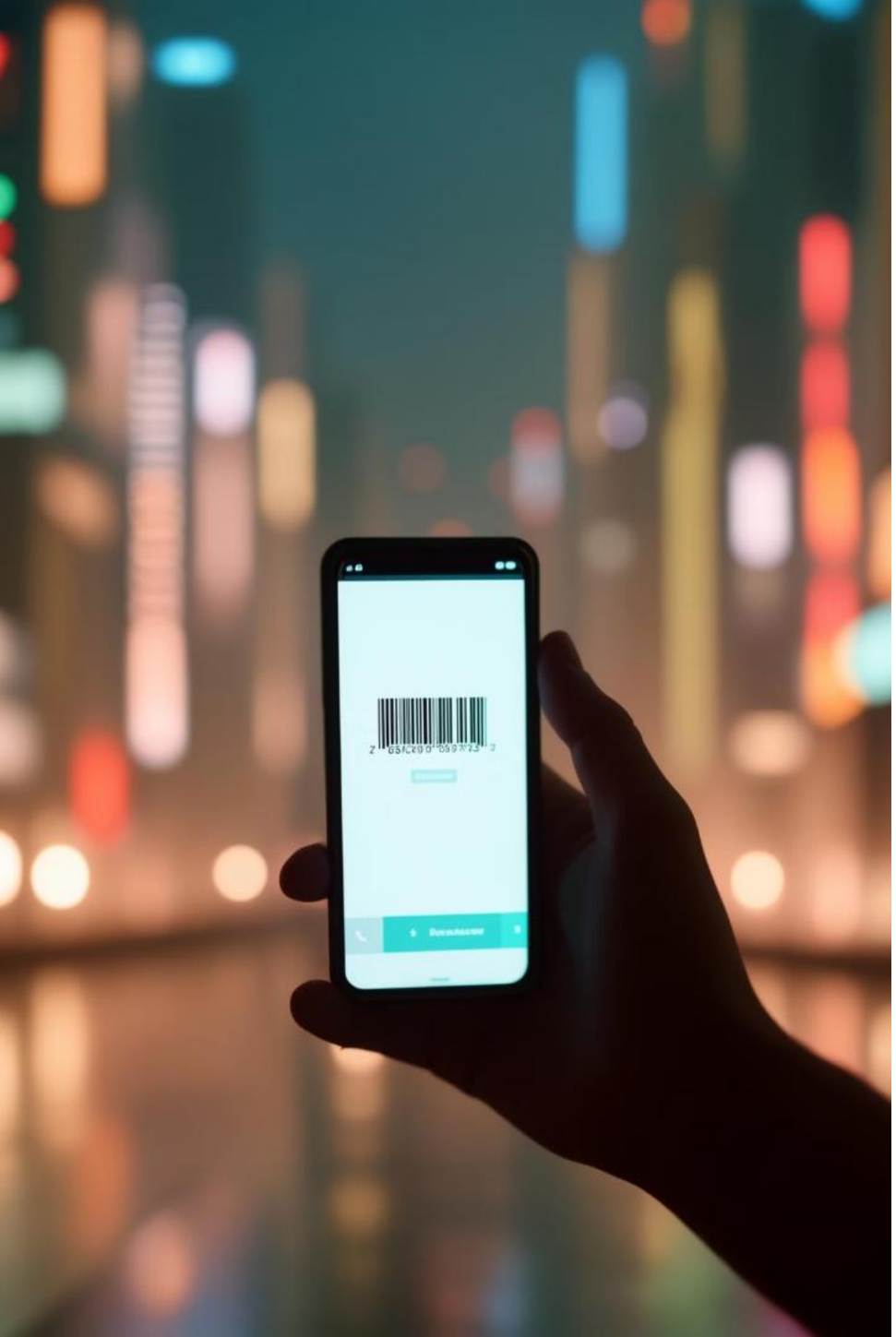
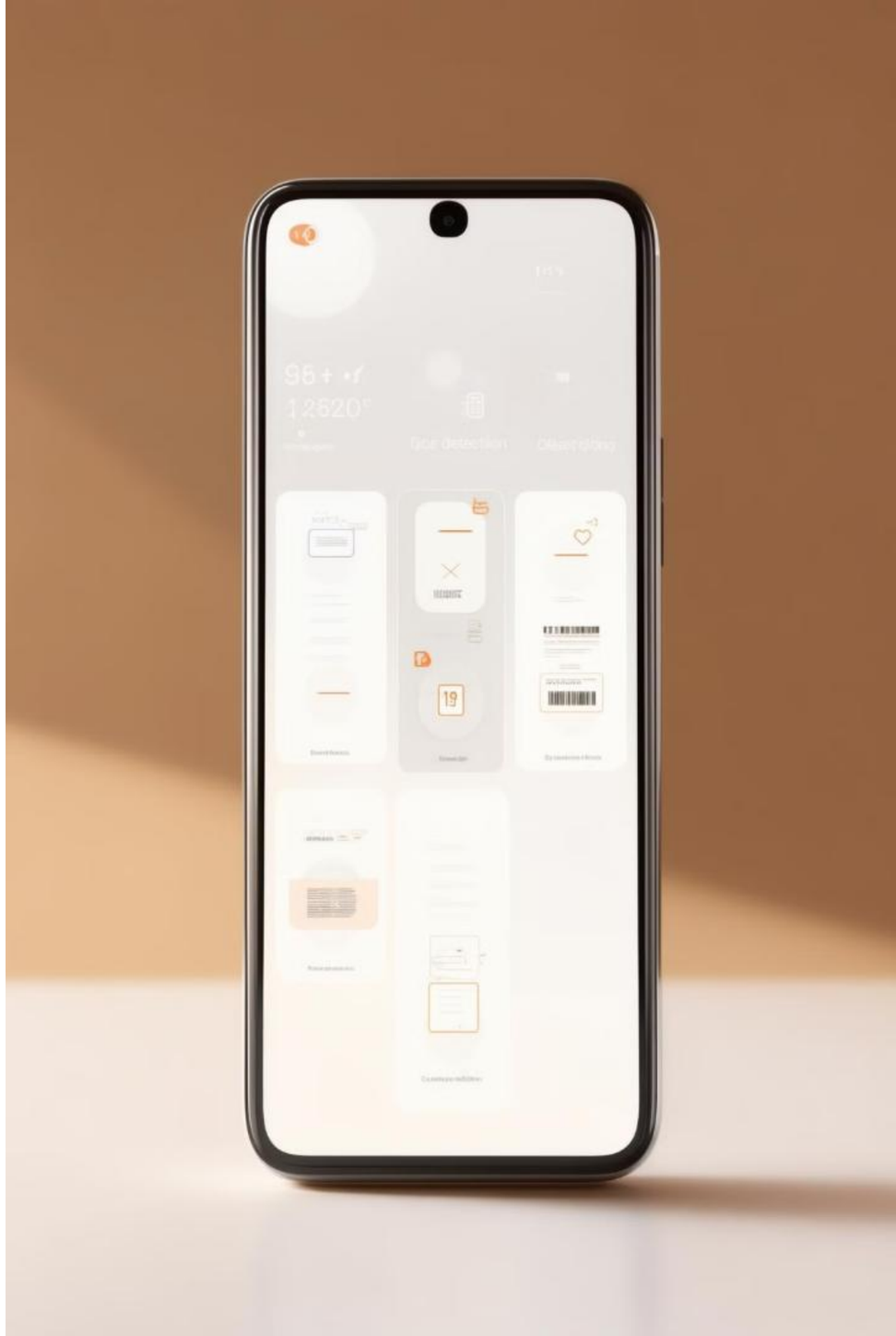


Firebase ML Kit for React Native

This presentation will guide you through integrating Firebase ML Kit into your React Native applications. We'll explore key features, installation steps, and code examples for text recognition, face detection, barcode scanning, object detection, and more.

Introduction to Firebase ML Kit

Firebase ML Kit is a powerful machine learning toolkit that empowers developers to integrate advanced features into their mobile applications. It offers a wide range of capabilities, including text recognition, face detection, barcode scanning, image classification, object detection and tracking, digital ink recognition, document scanning, pose detection, selfie segmentation, subject segmentation, and more.



Installing Firebase in a React Native Project

Install Firebase SDK

Use npm or Yarn to install the Firebase SDK and Firebase ML Kit:

```
npm install @react-native-firebase/app @react-native-firebase/ml
```

```
yarn add @react-native-firebase/app @react-native-firebase/ml
```

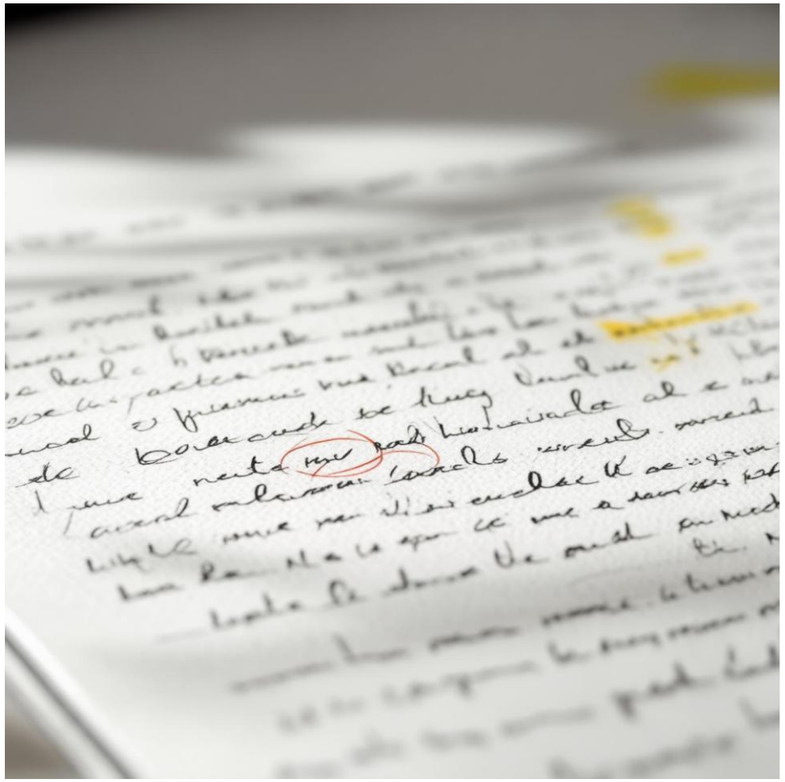
Configure Firebase for Android and iOS

Download the necessary configuration files from the Firebase Console and place them in the appropriate directories. Follow the specific instructions for Android and iOS.

Text Recognition (OCR)

Applications

- Automatic data entry from scanned documents
- Enhancing accessibility with text-to-speech conversion
- Real-time translation from images



Pros & Cons

- Works well with printed text
- Cloud-based option for better accuracy
- Struggles with handwritten text
- Requires internet for cloud-based recognition

Text Recognition (OCR)

Request camera access on Android

```
// Request camera access on Android
const requestCameraPermission = async () => {
  try {
    const granted = await PermissionsAndroid.request(
      PermissionsAndroid.PERMISSIONS.CAMERA,
      {
        title: "Camera Permission",
        message: "App needs access to your camera to take pictures.",
        buttonPositive: "OK",
      }
    );
    return granted === PermissionsAndroid.RESULTS.GRANTED;
  } catch (err) {
    console.warn(err);
    return false;
  }
};
```

Text recognition using Firebase ML Kit

```
// Text recognition using Firebase ML Kit
const recognizeText = async (imagePath) => {
  try {
    const result = await ml().cloudTextRecognizerProcessImage(imagePath);
    setRecognizedText(result.text);
  } catch (error) {
    console.error('Text recognition failed:', error);
    Alert.alert('Error', 'Failed to recognize text from image.');
  }
};
```

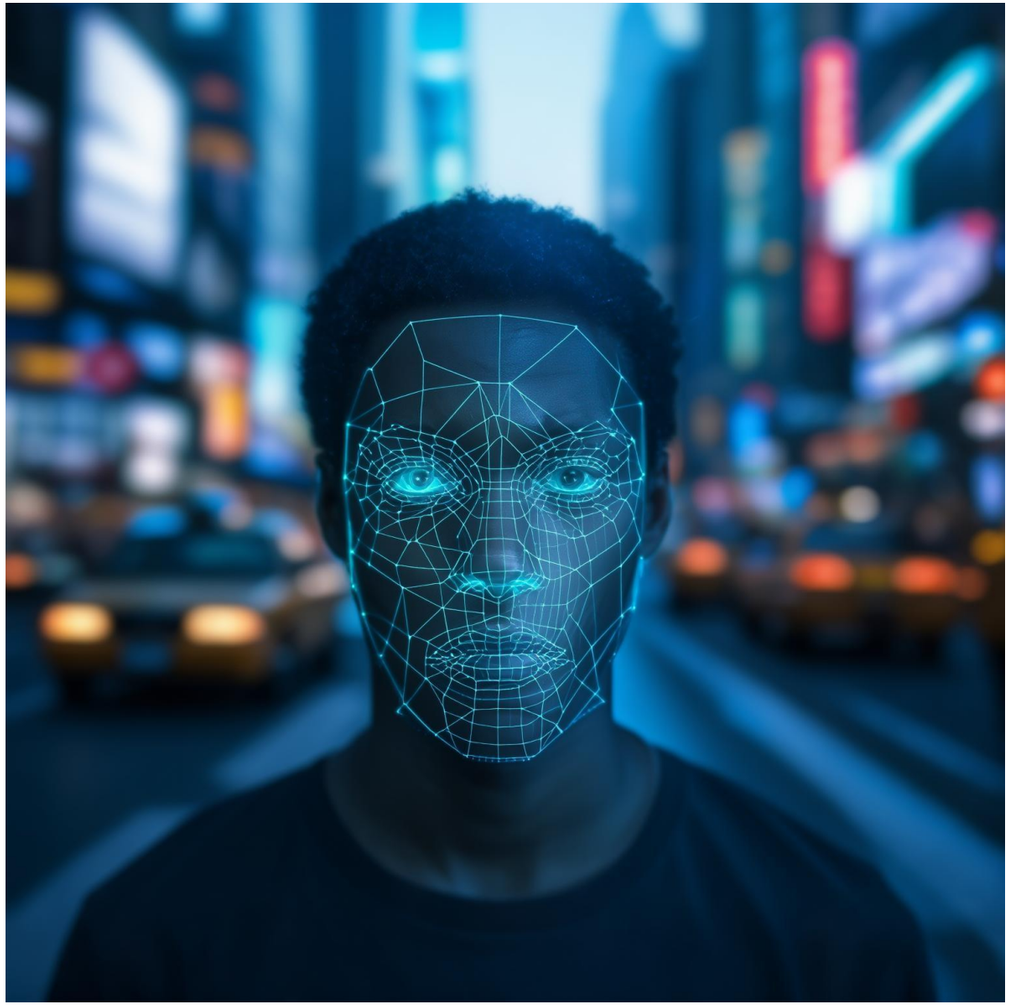
Face Detection

Applications

- Face authentication in security applications
- Applying facial filters in social media apps
- Detecting emotions for sentiment analysis

Pros & Cons

- Fast and efficient on-device processing
- Works offline
- Can struggle with occluded or low-light faces
- Limited deep emotion recognition



Face Detection

Request camera access on Android

```
const requestCameraPermission = async () => {
  try {
    const granted = await PermissionsAndroid.request(
      PermissionsAndroid.PERMISSIONS.CAMERA,
      {
        title: "Camera Permission",
        message: "App needs access to your camera to take pictures.",
        buttonPositive: "OK",
      }
    );
    return granted === PermissionsAndroid.RESULTS.GRANTED;
  } catch (err) {
    console.warn(err);
    return false;
  }
};
```

Face Detection with Firebase ML Kit

```
const detectFaces = async (imagePath) => {
  try {
    const faces = await ml().faceDetectorProcessImage(imagePath, {
      mode: 'accurate', // hoặc 'fast' nếu cần tốc độ nhanh hơn
      detectLandmarks: true,
      detectContours: true,
      classificationMode: 'all',
    });
    setFaceData(faces);
  } catch (error) {
    console.error('Face detection failed:', error);
    Alert.alert('Error', 'Failed to detect faces in image.');
  }
};
```

Barcode Scanning

Applications

- Inventory management in retail and warehouses
- Scanning QR codes for payments or authentication
- Ticket verification for events and transportation

Pros & Cons

- Fast and reliable for standard barcode formats
- Works offline
- May not work well with damaged or distorted barcodes
- Requires camera access

Barcode Scanning

Request camera access on Android

```
const requestCameraPermission = async () => {
  try {
    const granted = await PermissionsAndroid.request(
      PermissionsAndroid.PERMISSIONS.CAMERA,
      {
        title: "Camera Permission",
        message: "App needs access to your camera to scan barcodes.",
        buttonPositive: "OK",
      }
    );
    setCameraPermission(granted === PermissionsAndroid.RESULTS.GRANTED);
  } catch (err) {
    console.warn(err);
    setCameraPermission(false);
  }
};
```

Handling when scanning barcodes

```
const handleBarcodeScan = async (barcodes) => {
  if (barcodes && barcodes.length > 0) {
    try {
      const barcodeValue = barcodes[0].data;
      setBarcodeData(barcodeValue);
      Alert.alert('Barcode Scanned', `Value: ${barcodeValue}`);
    } catch (error) {
      console.error('Barcode scanning failed:', error);
      Alert.alert('Error', 'Failed to scan barcode.');
    }
  }
};
```

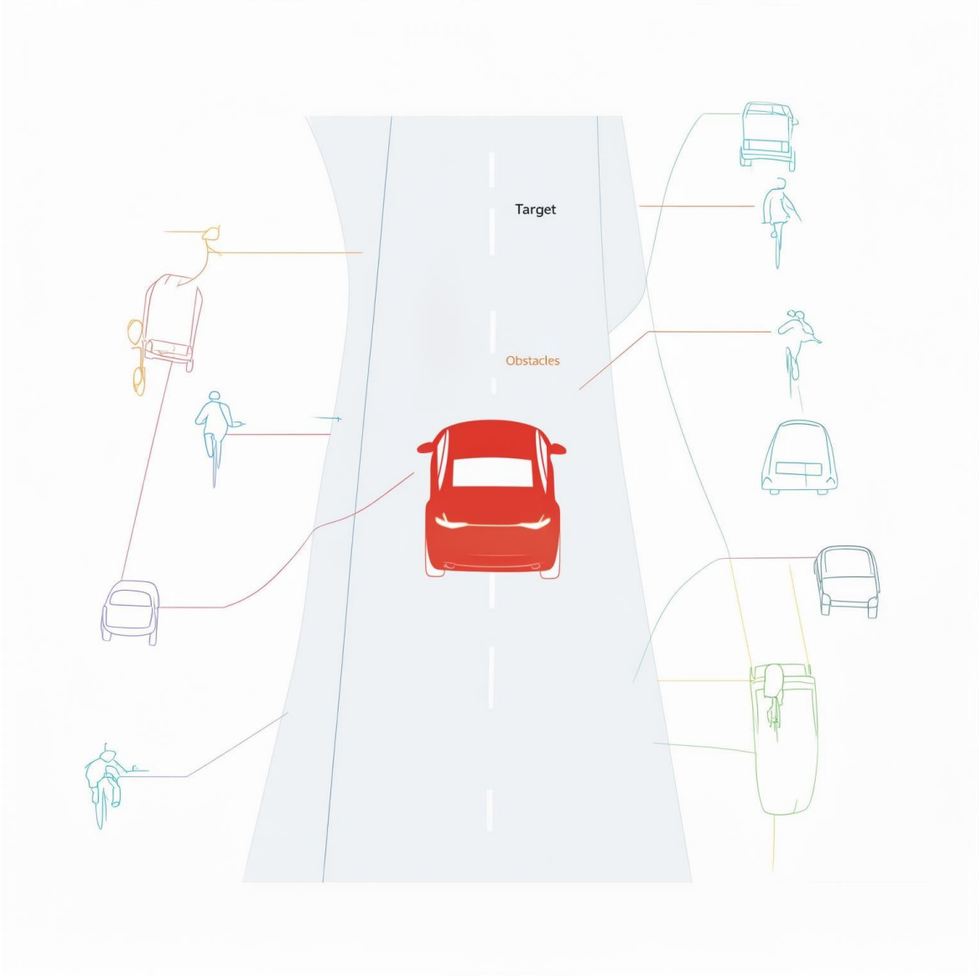
Object Detection and Tracking

Applications

- Identifying products for e-commerce recommendations
- Assisting visually impaired users with object identification
- Augmented reality applications for interactive experiences

Pros & Cons

- Can track multiple objects in real-time
- Works well for predefined categories
- Limited accuracy for custom objects
- Requires training data for new objects



Object Detection and Tracking

Request camera access on Android

```
const requestCameraPermission = async () => {
  try {
    const granted = await PermissionsAndroid.request(
      PermissionsAndroid.PERMISSIONS.CAMERA,
      {
        title: "Camera Permission",
        message: "App needs access to your camera for object detection.",
        buttonPositive: "OK",
      }
    );
    setCameraPermission(granted === PermissionsAndroid.RESULTS.GRANTED);
  } catch (err) {
    console.warn(err);
    setCameraPermission(false);
  }
};
```

Handling when detecting objects from camera

```
const handleObjectDetection = async (imagePath) => {
  try {
    const detectedObjects = await ml().objectDetectorProcessImage(imagePath, {
      mode: 'single-image', // Hoặc 'stream' nếu muốn theo dõi liên tục
      classifyObjects: true,
      multipleObjects: true,
    });

    setObjectsDetected(detectedObjects);
  } catch (error) {
    console.error('Object detection failed:', error);
    Alert.alert('Error', 'Failed to detect objects.');
  }
};
```

Digital Ink Recognition

Applications

- Handwriting recognition for note-taking apps
- Signature verification in financial applications
- Real-time digital whiteboard applications

Pros & Cons

- Works well for structured handwriting
- Cloud-based models improve accuracy
- Struggles with highly stylized or messy handwriting
- Requires internet for cloud-based processing

Digital Ink Recognition

On-screen drawing processing

```
const panResponder = PanResponder.create({
  onStartShouldSetPanResponder: () => true,
  onMoveShouldSetPanResponder: () => true,
  onPanResponderGrant: () => {
    setStrokes((prevStrokes) => [...prevStrokes, []]);
  },
  onPanResponderMove: (evt, gestureState) => {
    setStrokes((prevStrokes) => {
      const updatedStrokes = [...prevStrokes];
      updatedStrokes[updatedStrokes.length - 1].push({
        x: gestureState.moveX,
        y: gestureState.moveY,
      });
      return updatedStrokes;
    });
  },
});
```

Handwriting recognition processing

```
const recognizeInk = async () => {
  try {
    const formattedStrokes = strokes.map((stroke) => ({
      points: stroke.map((point) => ({ x: point.x, y: point.y, t: Date.now() })),
    }));

    const result = await ml().digitalInkRecognizerProcessInk({
      strokes: formattedStrokes,
      language: 'en-US', // Hỗ trợ nhiều ngôn ngữ khác nhau
    });

    setRecognizedText(result.text);
  } catch (error) {
    console.error('Digital ink recognition failed:', error);
    Alert.alert('Error', 'Failed to recognize handwriting.');
  }
};
```

Pose Detection

Applications

- Fitness tracking and exercise monitoring
- Motion analysis in sports performance
- Interactive gaming and augmented reality

Pros & Cons

Pros:

- Accurate body pose tracking with multiple key points
- Works well for fitness and rehabilitation applications
- Can operate on-device for real-time analysis

Cons:

- Struggles with occluded or partially visible bodies
- May not work well in low-light conditions
- Requires proper camera angle for best results

Pose Detection

Request camera access on Android

```
const requestCameraPermission = async () => {
  try {
    const granted = await PermissionsAndroid.request(
      PermissionsAndroid.PERMISSIONS.CAMERA,
      {
        title: "Camera Permission",
        message: "App needs access to your camera for pose detection.",
        buttonPositive: "OK",
      }
    );
    setCameraPermission(granted === PermissionsAndroid.RESULTS.GRANTED);
  } catch (err) {
    console.warn(err);
    setCameraPermission(false);
  }
};
```

Handling when detecting pose from camera

```
const handlePoseDetection = async (imagePath) => {
  try {
    const detectedPoses = await ml().poseDetectorProcessImage(imagePath, {
      mode: 'accurate', // Hoặc 'fast' nếu muốn tốc độ cao hơn
    });

    setPoseData(detectedPoses);
  } catch (error) {
    console.error('Pose detection failed:', error);
    Alert.alert('Error', 'Failed to detect poses.');
  }
};
```

Selfie Segmentation

Applications

- Virtual background replacement in video calls
- Real-time beauty and AR filter applications
- Green screen effects for content creation

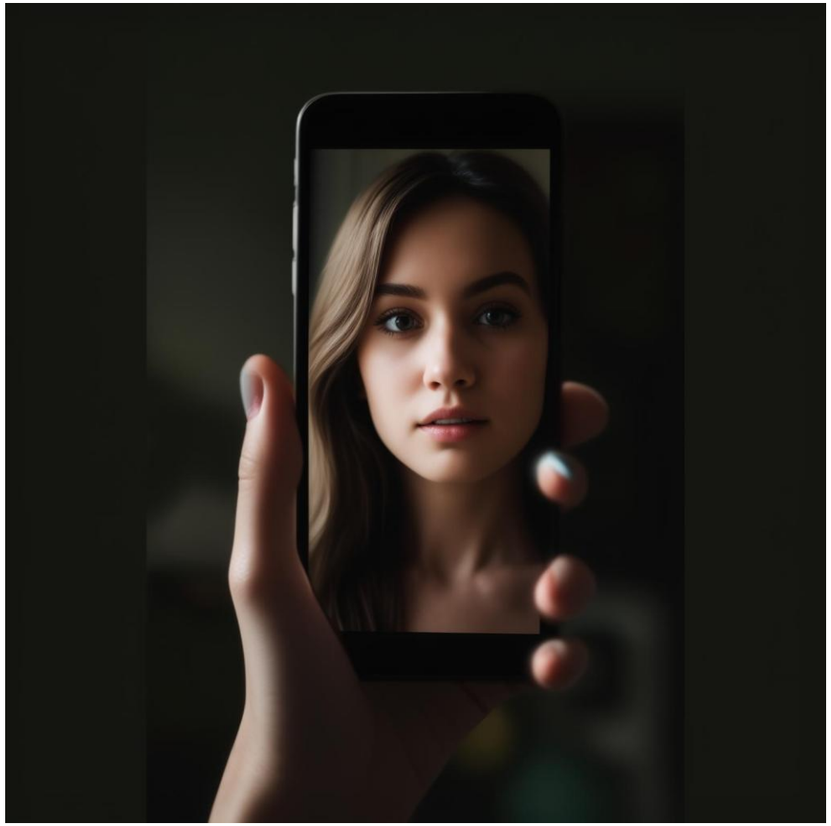
Pros & Cons

Pros:

- Fast and efficient background removal
- Works well with most selfie images
- Supports real-time video processing

Cons:

- Struggles with hair details and overlapping objects
- May require good lighting for best accuracy
- Can be resource-intensive on mobile devices



Selfie Segmentation

Request camera access on Android

```
const requestCameraPermission = async () => {
  try {
    const granted = await PermissionsAndroid.request(
      PermissionsAndroid.PERMISSIONS.CAMERA,
      {
        title: "Camera Permission",
        message: "App needs access to your camera for selfie segmentation.",
        buttonPositive: "OK",
      }
    );
    return granted === PermissionsAndroid.RESULTS.GRANTED;
  } catch (err) {
    console.warn(err);
    return false;
  }
};
```

Open camera to take photo

```
const openCamera = async () => {
  const hasPermission = await requestCameraPermission();
  if (!hasPermission) {
    Alert.alert("Permission Denied", "You need to grant camera permission to use this feature.");
    return;
  }

  launchCamera({ mediaType: 'photo', quality: 1 }, (response) => {
    if (!response.didCancel && !response.error) {
      setImageUri(response.assets[0].uri);
      processSelfieSegmentation(response.assets[0].uri);
    }
  });
};
```

Handling Selfie Segmentation with Firebase ML Kit

```
const processSelfieSegmentation = async (imagePath) => {
  try {
    const segmentationMask = await ml().selfieSegmenterProcessImage(imagePath, {
      mode: 'accurate', // Hoặc 'fast' nếu cần tốc độ cao hơn
    });

    setSegmentedImage(segmentationMask);
  } catch (error) {
    console.error('Selfie segmentation failed:', error);
    Alert.alert('Error', 'Failed to segment selfie.');
  }
};
```

Subject Segmentation

Applications

- Identifying and isolating objects for image editing
- AI-powered photography enhancements
- Automatic object cutouts for e-commerce

Pros & Cons

Pros:

- Works well for detecting and segmenting key subjects
- Useful for automatic background removal
- Can be applied to various industries like retail and media

Cons:

- Struggles with complex backgrounds
- May require fine-tuning for high accuracy
- Processing speed depends on image resolution

Subject Segmentation

Request camera access on Android

```
const requestCameraPermission = async () => {
  try {
    const granted = await PermissionsAndroid.request(
      PermissionsAndroid.PERMISSIONS.CAMERA,
      {
        title: "Camera Permission",
        message: "App needs access to your camera for selfie segmentation.",
        buttonPositive: "OK",
      }
    );
    return granted === PermissionsAndroid.RESULTS.GRANTED;
  } catch (err) {
    console.warn(err);
    return false;
  }
};
```

Open camera to take photo

```
const openCamera = async () => {
  const hasPermission = await requestCameraPermission();
  if (!hasPermission) {
    Alert.alert("Permission Denied", "You need to grant camera permission to use this feature.");
    return;
  }

  launchCamera({ mediaType: 'photo', quality: 1 }, (response) => {
    if (!response.didCancel && !response.error) {
      setImageUri(response.assets[0].uri);
      processSelfieSegmentation(response.assets[0].uri);
    }
  });
};
```

Handling Subject Segmentation with Firebase ML Kit

```
const processSubjectSegmentation = async (imagePath) => {
  try {
    const segmentationMask = await ml().subjectSegmenterProcessImage(imagePath, {
      mode: 'accurate', // Hoặc 'fast' nếu cần tốc độ cao hơn
    });

    setSegmentedImage(segmentationMask);
  } catch (error) {
    console.error('Subject segmentation failed:', error);
    Alert.alert('Error', 'Failed to segment subject.');
  }
};
```

Document Scanning

Applications

- Digitizing paper documents into text or PDF
- Automatic form recognition and processing
- Optical Character Recognition (OCR) for document extraction

Pros & Cons

Pros:

- Enhances document clarity and readability
- Converts physical documents into digital text
- Can work on-device without an internet connection

Cons:

- Accuracy depends on text quality and alignment
- Struggles with handwritten or blurred text
- Requires proper lighting for best performance

Document Scanning

Request camera access on Android

```
const requestCameraPermission = async () => {
  try {
    const granted = await PermissionsAndroid.request(
      PermissionsAndroid.PERMISSIONS.CAMERA,
      {
        title: "Camera Permission",
        message: "App needs access to your camera for selfie segmentation.",
        buttonPositive: "OK",
      }
    );
    return granted === PermissionsAndroid.RESULTS.GRANTED;
  } catch (err) {
    console.warn(err);
    return false;
  }
};
```

Open camera to take photo

```
const openCamera = async () => {
  const hasPermission = await requestCameraPermission();
  if (!hasPermission) {
    Alert.alert("Permission Denied", "You need to grant camera permission to use this feature.");
    return;
  }

  launchCamera({ mediaType: 'photo', quality: 1 }, (response) => {
    if (!response.didCancel && !response.error) {
      setImageUri(response.assets[0].uri);
      processSelfieSegmentation(response.assets[0].uri);
    }
  });
};
```

Handling document scanning with Firebase ML Kit

```
const processDocumentScanning = async (imagePath) => {
  try {
    const recognizedText = await ml().cloudTextRecognizerProcessImage(imagePath);
    setScannedText(recognizedText.text);
  } catch (error) {
    console.error('Document scanning failed:', error);
    Alert.alert('Error', 'Failed to scan the document.');
  }
};
```

Overview of @tensorflow/tfjs-react-native

What it is

A library that enables TensorFlow.js to run on React Native applications. This allows Machine Learning models to be executed directly on mobile devices, without relying on a server.

Key Applications

- Image Recognition
- Speech Processing
- Object Detection

Strengths & Weaknesses

1 **Runs directly on mobile devices**

No need for an internet connection. This enhances processing speed and data privacy.

2 **Supports WebGL & GPU acceleration**

Improves computational performance when hardware supports it.

3 **Integrates well with other libraries**

Makes image and video processing easier.

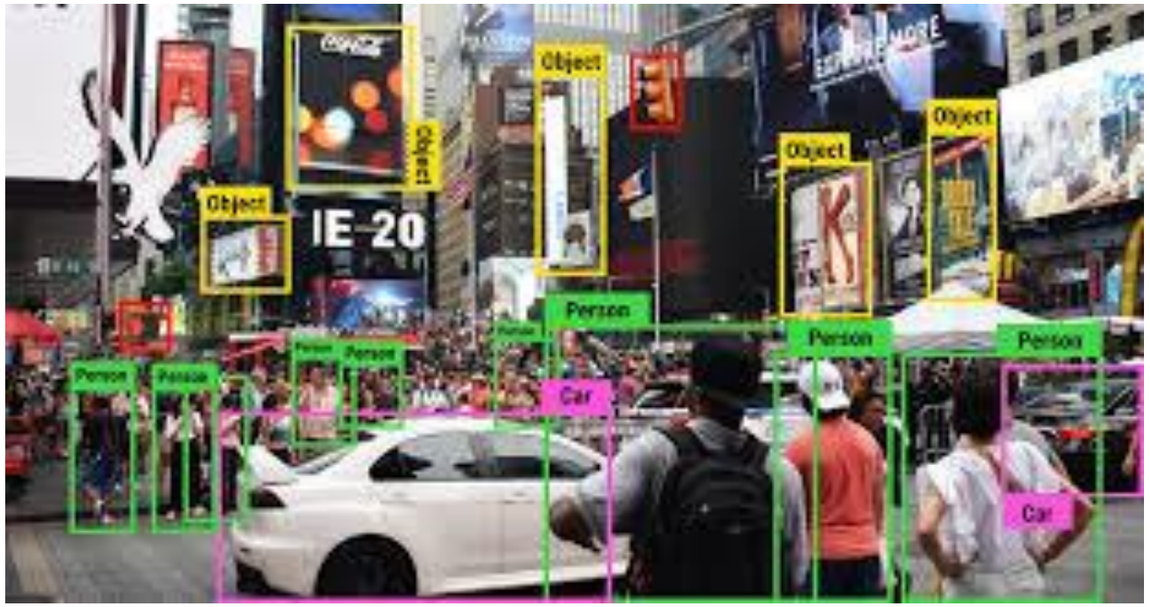
4 **No GPU acceleration support on iOS**

Due to WebGL limitations.

5 **Lower performance compared to TensorFlow Lite**

Not optimized with C++ like TFLite.

Image Recognition



MobileNet

Object recognition in images (lightweight, fast)

EfficientNet

Optimized version of MobileNet with higher accuracy

PoseNet

Human pose detection (AI fitness, AR, etc.)

YOLO

Multi-object detection in images/videos (slow on mobile)

Audio Processing

Audio Processing

- Speech Commands
- DeepSpeech

Text Processing

BERT

Text classification, chatbots, smart search

Text Toxicity

Detecting toxic content in messages and comments

GPT-2

Small AI-generated text (small version of GPT)

Running a Model in React Native

Install Dependencies

```
npm install @tensorflow/tfjs @tensorflow/tfjs-react-native @react-native-async-storage/async-storage
npm install react-native-image-picker
npx pod-install # Run on iOS
```

https://www.tensorflow.org/js/tutorials/applications/react_native?hl=vi

Running a Model in React Native

```
import React, { useEffect, useState } from "react";
import { View, Text, Button, Image, ActivityIndicator } from "react-native";
import * as tf from "@tensorflow/tfjs";
import "@tensorflow/tfjs-react-native";
import * as ImagePicker from "react-native-image-picker";

const App = () => {
  const [model, setModel] = useState(null);
  const [image, setImage] = useState(null);
  const [prediction, setPrediction] = useState(null);
  const [loading, setLoading] = useState(true);

  // Load MobileNet model
  useEffect(() => {
    const loadModel = async () => {
      await tf.ready();
      const mobilenet = await tf.loadLayersModel(
        "https://storage.googleapis.com/tfjs-models/tfjs/mobilenet_v1_1.0_224/
model.json"
      );
      setModel(mobilenet);
      setLoading(false);
    };
```

```
    loadModel();
  }, []);

  // Select an image from the gallery
  const pickImage = () => {
    ImagePicker.launchImageLibrary({ mediaType: "photo" }, (response) => {
      if (response.didCancel) return;
      setImage(response.assets[0].uri);
    });
  };

  // Run image classification
  const classifyImage = async () => {
    if (!model || !image) return;

    setLoading(true);
    const imgTensor = await imageToTensor(image);
    const prediction = model.predict(imgTensor);
    const result = prediction.dataSync();
    setPrediction(result);
    setLoading(false);
```

```
  // Convert image to tensor
  const imageToTensor = async (uri) => {
    const imageAsset = await fetch(uri);
    const imageBlob = await imageAsset.blob();
    const imageArrayBuffer = await imageBlob.arrayBuffer();
    const raw = new Uint8Array(imageArrayBuffer);
    const imageTensor = tf.tensor3d(raw, [1, 224, 224, 3]);
    return imageTensor;
  };

  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      {loading && <ActivityIndicator size="large" color="#0000ff" />}
      {image && <Image source={{ uri: image }} style={{ width: 200, height:
200 }} />}
      <Button title="Select Image" onPress={pickImage} />
      <Button title="Classify Image" onPress={classifyImage} disabled={!image}
/>
      {prediction && <Text>Result: {JSON.stringify(prediction)}</Text>}
    </View>
  );
};
```

https://snack.expo.dev/@taitv/tensorflow_tfjs-